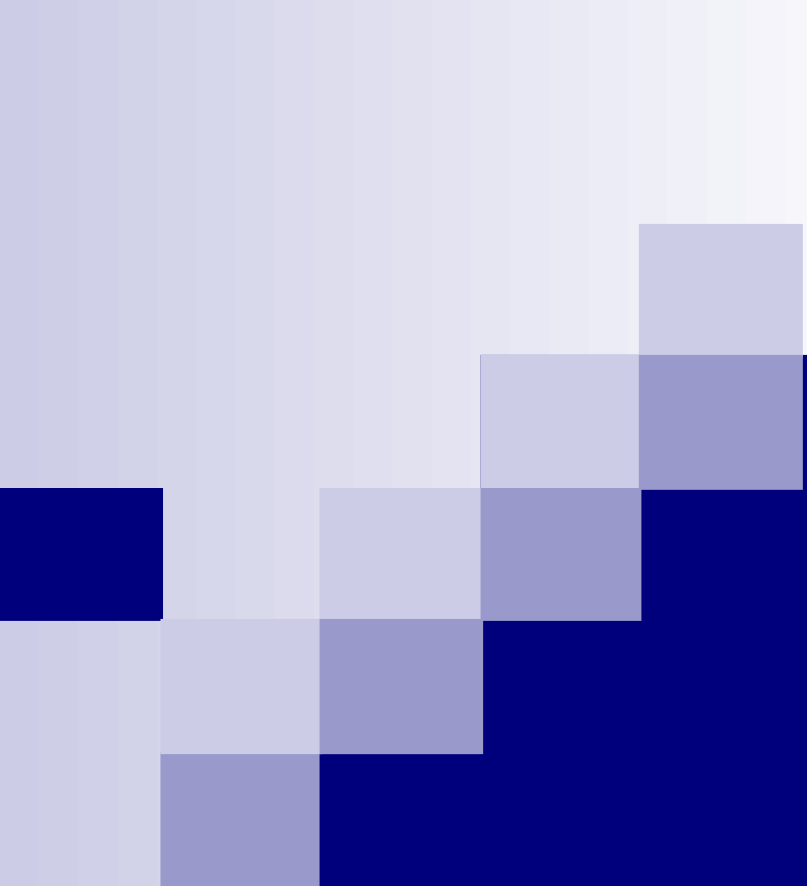




Homework Assignment #1:

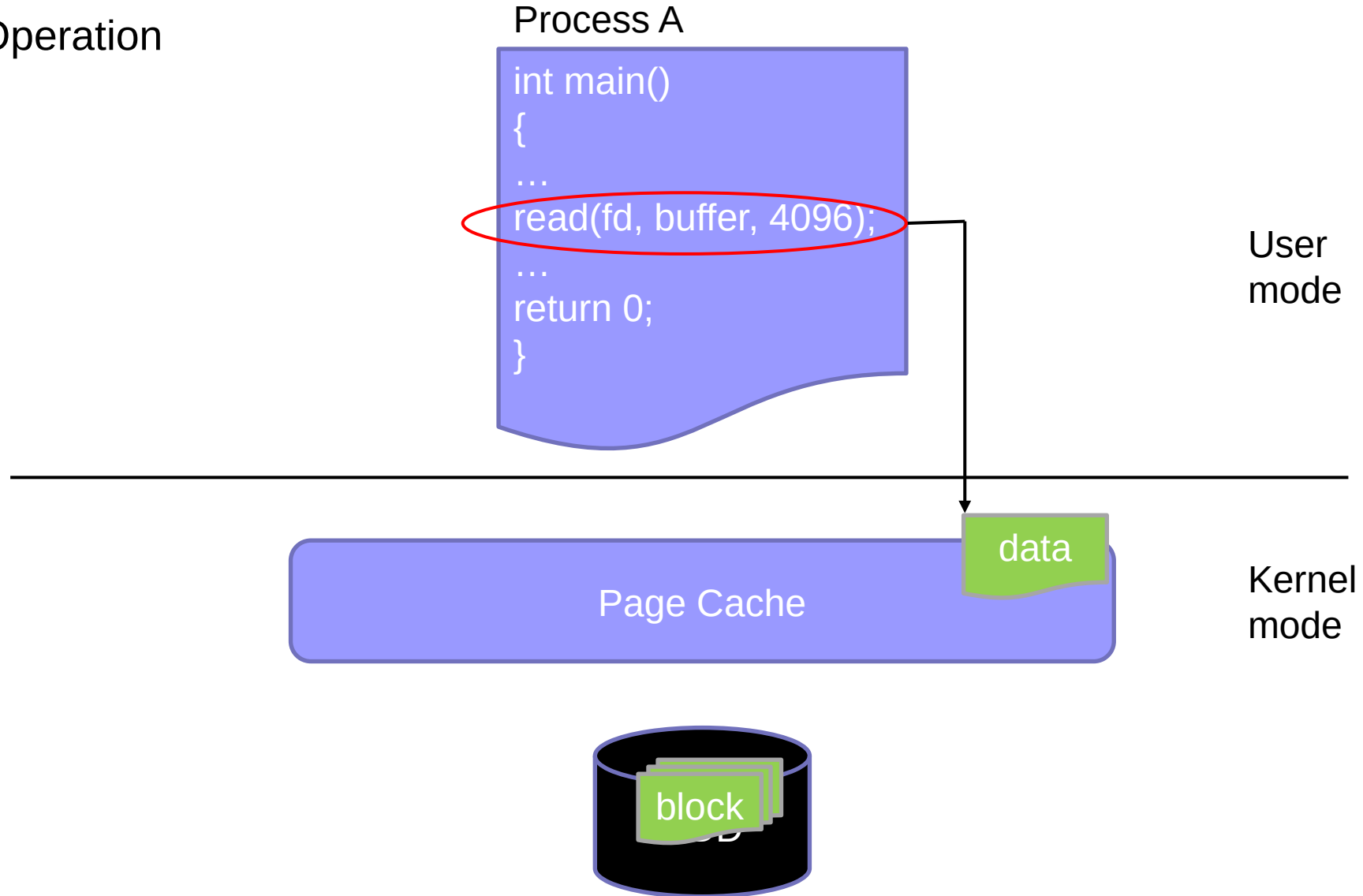
Disk IO Performance under Sequential and Random Workloads by Using System Call, C Library and MMAP Interfaces

Outline

- **File I/O**
- C-library : fopen() 、 fread() 、 fwrite() 、 fclose() 、 fseek()
- Homework Assignment #1_1
- System call : open() 、 read() 、 write() 、 close() 、 lseek()
- Homework Assignment #1_2
- Memory-mapped file
- Homework Assignment #1_3
- Homework Assignment #1_4

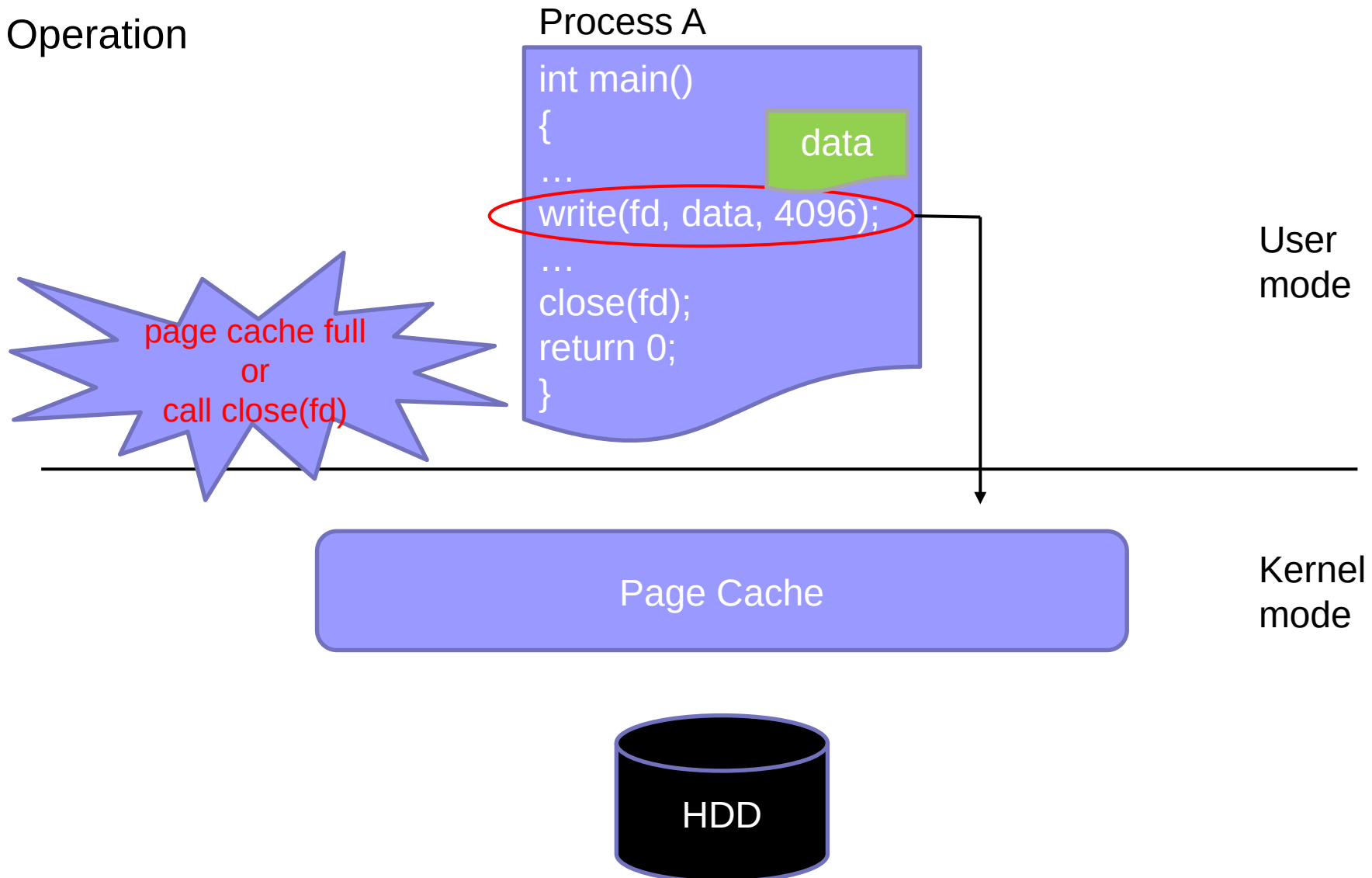
File Input / Output

- Read Operation



File Input / Output

- Write Operation



Outline

- File I/O
- **C-library : fopen() 、 fread() 、 fwrite() 、 fclose() 、 fseek()**
- Homework Assignment #1_1
- System call : open() 、 read() 、 write() 、 close() 、 lseek()
- Homework Assignment #1_2
- Memory-mapped file
- Homework Assignment #1_3
- Homework Assignment #1_4

Standard C library

■ fopen()

```
#include <stdio.h>
```

```
FILE *fopen ( const char *filename, const char *mode );
```

```
EX: FILE *f1;  
     f1 = fopen ( "XXX.txt", "w" );
```

mode:

"r"	read: Open file for input operations. The file must exist.
"w"	write: Create an empty file for output operations. If a file with the same name already exists, its contents are discarded and the file is treated as a new empty file.
"a"	append: Open file for output at the end of a file. Output operations always write data at the end of the file, expanding it. The file is created if it does not exist.

return value: Successful: the function returns a pointer to a **FILE** object. Otherwise: a null pointer is returned.

Standard C library(cont.)

■ fread()

```
#include <stdio.h>
```

```
size_t fread ( void * ptr, size_t size, size_t count, FILE * stream );
```

```
EX: fread ( buffer, sizeof(char), 4, f1 );
```

ptr: Pointer to a block of memory with a size of at least (**size*count**) bytes.

size: Size, in bytes, of each element to be read. **size_t** is an unsigned integral type.

count: Number of elements, each one with a size of **size** bytes.

stream: Pointer to a **FILE** object that specifies an input stream.

return value: The total number of elements successfully read is returned. If this number differs from the count parameter, the proper indicator is set, which can be checked with **ferror** and **feof**.

Standard C library(cont.)

■ fwrite()

```
#include <stdio.h>
```

```
size_t fwrite ( const void * ptr, size_t size, size_t count, FILE * stream );
```

```
EX: fwrite ( buffer, sizeof(char), 4, f1 );
```

ptr: Pointer to the array of elements to be written, converted to a const void*.

size: Size, in bytes, of each element to be written. **size_t** is an unsigned integral type.

count: Number of elements, each one with a size of **size** bytes.

stream: Pointer to a **FILE** object that specifies an output stream.

return value: The total number of elements successfully written is returned. If this number differs from the count parameter, a writing error prevented the function from completing. In this case, the error indicator (**ferror**) will be set for the stream.

Standard C library(cont.)

■ fclose()

```
#include <stdio.h>
```

```
int fclose ( FILE * stream );
```

```
EX: fclose (f1);
```

stream: Pointer to a **FILE** object that specifies the stream to be closed.

return value: If the stream is successfully closed, a zero value is returned. On failure, EOF is returned.

Standard C library(cont.)

■ fseek()

```
#include <stdio.h>
```

```
int fseek(FILE *stream, long int offset, int whence)
```

```
EX: fseek (f1,4096,SEEK_SET);
```

stream: Pointer to a FILE object that identifies the stream.

long: Number of bytes to offset from origin.

whence: Position used as reference for the offset. It is specified by one of the following constants defined in <stdio.h> exclusively to be used as arguments for this function:

Constant	Reference position
SEEK_SET	Beginning of file
SEEK_CUR	Current position of the file pointer
SEEK_END	End of file *

return value: If successful, the function returns zero.

Otherwise, it returns non-zero value.

If a read or write error occurs, the error indicator (ferror) is set.

Homework Assignment #1_1(cont.)

- fsync()

- fsync() transfers all modified in-core data of the file referred to by the file descriptor fd to the disk device so that all changed information can be retrieved even if the system crashes or is rebooted. This includes writing through or flushing a disk cache if present. The call blocks until the device reports that the transfer has completed.

```
#include <unistd.h>
```

```
int fsync(int fd);
```

- The function fileno() examines the argument stream and returns its integer descriptor.

```
int fileno(FILE *stream);
```

```
Ex : fsync(fileno(fp))
```

Outline

- File I/O
- C-library : fopen() 、 fread() 、 fwrite() 、 fclose() 、 fseek()
- **Homework Assignment #1_1**
- System call : open() 、 read() 、 write() 、 close() 、 lseek()
- Homework Assignment #1_2
- Memory-mapped file
- Homework Assignment #1_3
- Homework Assignment #1_4

Homework Assignment #1_1

- Write a C program that creates a **100MB file** on your local disk and measures the execution time of five different operations using the **C standard I/O library interface** (e.g., fopen(), fread(), fwrite(), fseek(), and fclose().)
- **Implement each of the following operations as a separate function**
 - ☐ Sequential Read
 - ☐ Sequential Write
 - ☐ Random Read
 - ☐ Random Buffered Write (Single Sync)
 - ☐ Random Synchronous Write

Homework Assignment #1_1

■ Sequential Read

- ☐ Read the file sequentially from beginning to end.
- ☐ Read **4KB of data per operation**.
- ☐ Continue until the **entire 100MB file has been read**.

■ Sequential Write

- ☐ Overwrite the file with **100MB of new data** using sequential writes.
- ☐ Write **2KB of data per operation** from beginning to end.
- ☐ After completing all writes, call **fsync()** to ensure that the data is flushed to disk.

Homework Assignment #1_1(cont.)

■ **time()**

- time() function is provided by time.h library.
- Calculates the current calendar time and encodes it into time_t format.

```
#include<time.h>
```

```
time_t time(time_t *t);
```

```
ex. : time(NULL);
```

Homework Assignment #1_1(cont.)

■ rand()

- rand() function is provided by stdlib.h library.
- Returns a pseudo-random number in the range of 0 to RAND_MAX.

```
#include<stdlib.h>
```

```
int rand(void);
```

```
ex. : rand();
```

Homework Assignment #1_1(cont.)

■ **srand()**

- srand() function is provided by stdlib.h library.
- This function seeds the random number generator used by the function rand().

```
#include<stdlib.h>
```

```
void srand(unsigned int seed);
```

```
ex. : srand(time(NULL));
```

Homework Assignment #1_1

■ Random Offset Generation

- ☐ Use rand() to generate a random block index.
- ☐ Initialize the random seed once using: srand(time(NULL));
- ☐ Ensure the selected offset is **4KB-aligned**.
- ☐ The offset must be within the 100MB file range.

■ Hint

- ☐ Call srand() in main().
- ☐ Before each random write or read, call (rand()%block numbers)*block size.

Homework Assignment #1_1

■ Random Read

- Repeat the following operation **50,000 times**:
 - Select a **4KB-aligned offset** uniformly at random within the file.
 - Seek to the selected offset.
 - Read **4KB of data** from that position.

■ Random Buffered Write (Single Sync)

- Repeat the following operation **50,000 times**:
 - Select a **4KB-aligned offset** uniformly at random within the file.
 - Seek to the selected offset.
 - Write **2KB of data** at that position.

Homework Assignment #1_1

■ Random Synchronous Write

- Repeat the following operation **50,000 times**:
 - Select a **4KB-aligned offset** uniformly at random within the file.
 - Seek to the selected offset.
 - Write **2KB of data** at that position.
 - Call fsync() after each write.

- The execution time of each workload must be measured independently.
 - Use gettimeofday() (or a similar high-resolution timing function).
 - The timing interval should include only the workload execution.
 - Exclude file creation time from the measurement.
 - Report the total elapsed time for each workload.

Homework Assignment #1_1(cont.)

- gettimeofday()
 - Get the time as well as a timezone.

```
#include <sys/time.h>
```

```
int gettimeofday (struct timeval *tv, struct timezone *tz );
```

```
struct timeval {  
    time_t tv_sec;           /* seconds */  
    suseconds_t tv_usec;    /* microseconds */  
};
```

```
struct timezone {  
    int tz_minuteswest;     /* minutes west of Greenwich */  
    int tz_dsttime;         /* type of DST correction */  
};
```

Homework Assignment #1_1(cont.)

- EX:

```
int main()
{
    struct timeval start;
    struct timeval end;

    unsigned long diff;
    gettimeofday(&start,NULL);
    delay(10);
    gettimeofday(&end,NULL);
    diff = 1000000 * (end.tv_sec-start.tv_sec)+ end.tv_usec-start.tv_usec;
    printf("the difference is %ld (us)\n", diff);
    return 0;
}
```

Homework Assignment #1_1(cont.)

- If you want to take a look at the total available as well as used memory in the system. You can use **free** command.
- To clean the Linux buffer/cache, you can use **echo 3 > /proc/sys/vm/drop_caches** command. Notice that, you should run it in **root** mode.
- Ex:

```
root@oslab-ASUSPRO-D840MB-M840MB:/home/oslab# free
              total        used        free      shared  buff/cache   available
Mem:      16195052     836396    14732860     136168     625796    14946580
Swap:      2097148       84224     2012924
root@oslab-ASUSPRO-D840MB-M840MB:/home/oslab# echo 3 > /proc/sys/vm/drop_caches
root@oslab-ASUSPRO-D840MB-M840MB:/home/oslab#
```

Outline

- File I/O
- C-library : fopen() 、 fread() 、 fwrite() 、 fclose() 、 fseek()
- Homework Assignment #1_1
- **System call : open() 、 read() 、 write() 、 close() 、 lseek()**
- Homework Assignment #1_2
- Memory-mapped file
- Homework Assignment #1_3
- Homework Assignment #1_4

Linux system call interface

■ open()

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
```

```
int open ( const char *filename, int flags );
int open ( const char *filename, int flags, mode_t mode );
```

```
EX: f1 = open ( "hello.txt", O_WRONLY | O_TRUNC );
```

flags:

O_RDONLY	Read only.
O_WRONLY	Write only.
O_RDWR	Read and write.
O_CREAT	Create file if it doesn't exist.
O_TRUNC	If the file already exists and the open mode allows writing (i.e., is O_RDWR or O_WRONLY) it will be truncated to length 0.

return value: Return the file descriptor, or -1 is returned and **errno** is set appropriately.

Linux system call interface(cont.)

■ read()

```
#include<unistd.h>
```

```
ssize_t read(int fd, void *buf, size_t count);
```

```
EX: read(f1, buffer, 4);
```

fd: The file descriptor of where to read the input.

buf: A character array where the read content will be stored.

count: The number of bytes to read before truncating the data. If the data to be read is smaller than nbytes, all data is saved in the buffer.

return value: Upon successful completion, it shall return a non-negative integer indicating the number of bytes actually read. Otherwise, the functions shall return -1 and set ***errno*** to indicate the error.

Linux system call interface(cont.)

■ write()

```
#include<unistd.h>
```

```
ssize_t write(int fd, const void *buf, size_t count);
```

```
EX: write(f1, buffer, 4);
```

fd: The file descriptor of where to write the input.

buf: A pointer to a buffer of at least nbytes bytes, which will be written to the file.

count: The number of bytes to write. If smaller than the provided buffer, the output is truncated.

return value: On success, the number of bytes written is returned. On error, -1 is returned, and **errno** is set to indicate the cause of the error.

Linux system call interface(cont.)

- **close()**

```
#include<unistd.h>
```

```
int close(int fd);
```

```
EX: close(f1);
```

fd: The file descriptor to be closed.

return value: returns zero on success. On error, -1 is returned, and **errno** is set appropriately.

Linux system call interface(cont.)

■ lseek()

```
#include <sys/types.h>
#include <unistd.h>
```

```
off_t lseek(int fd, off_t offset, int whence);
```

```
EX: write(f1, 4096, SEEK_SET);
```

fd: The file descriptor of where to change offset.

offset: Number of bytes to offset from origin.

whence: Position used as reference for the offset.

return value: Upon successful completion, lseek() returns the resulting offset location as measured in bytes from the beginning of the file. On error, the value (off_t) -1 is returned and errno is set to indicate the error.

Outline

- File I/O
- C-library : fopen() 、 fread() 、 fwrite() 、 fclose() 、 fseek()
- Homework Assignment #1_1
- System call : open() 、 read() 、 write() 、 close() 、 lseek()
- **Homework Assignment #1_2**
- Memory-mapped file
- Homework Assignment #1_3
- Homework Assignment #1_4

Homework Assignment #1_2

- Write a C program that creates a **100MB file** on your local disk and measures the execution time of five different operations using the **system call I/O interface**(e.g., open() ,read(), write(), lseek(), and close()).
- Implement each of the following operations as a separate function
 - ☐ Sequential Read
 - ☐ Sequential Write
 - ☐ Random Read
 - ☐ Random Buffered Write (Single Sync)
 - ☐ Random Synchronous Write

Homework Assignment #1_2

■ Sequential Read

- ☐ Read the file sequentially from beginning to end.
- ☐ Read **4KB of data per operation**.
- ☐ Continue until the entire 100MB file has been read.

■ Sequential Write

- ☐ Overwrite the file with **100MB of new data** using sequential writes.
- ☐ Write **2KB of data per operation** from beginning to end.
- ☐ After completing all writes, call `fsync()` to ensure that the data is flushed to disk.

Homework Assignment #1_2

■ Random Read

- Repeat the following operation **50,000 times**:
 - Select a **4KB-aligned offset** uniformly at random within the file.
 - Seek to the selected offset using `lseek()`.
 - Read **4KB of data** from that position.

■ Random Buffered Write (Single Sync)

- Repeat the following operation **50,000 times**:
 - Select a **4KB-aligned offset** uniformly at random within the file.
 - Seek to the selected offset using `lseek()`.
 - Write **2KB of data** at that position.
- After all writes have been completed, call `fsync()` once.

Homework Assignment #1_2

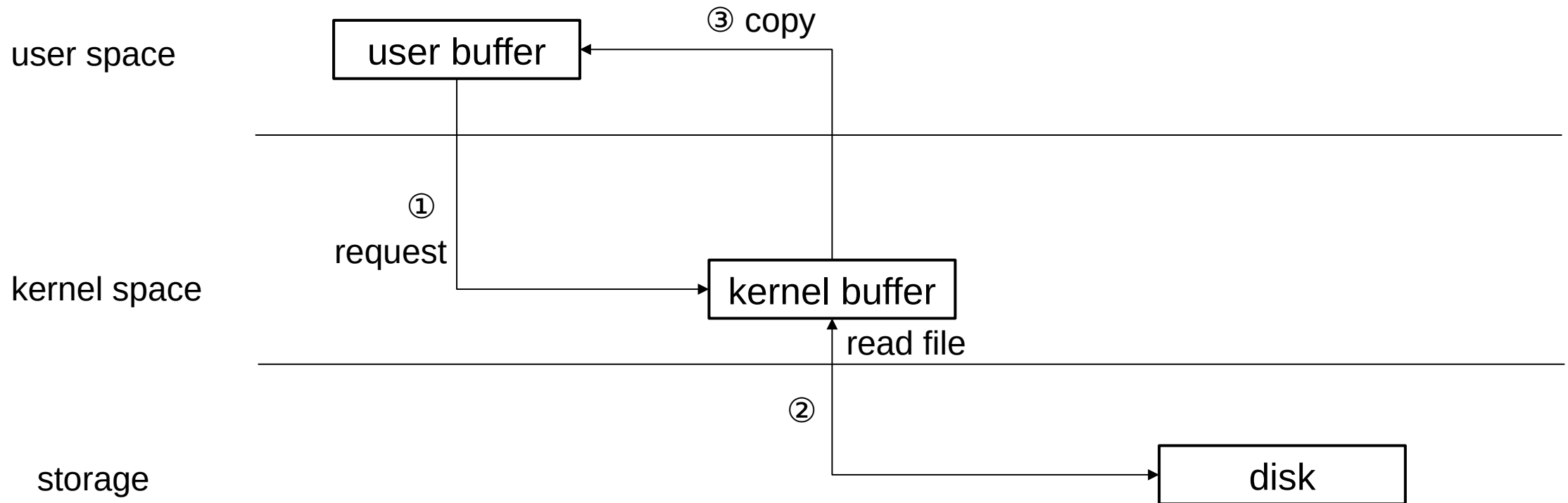
- Random Synchronous Write
 - Repeat the following operation **50,000 times**:
 - Select a **4KB-aligned offset** uniformly at random within the file.
 - Seek to the selected offset using `lseek()`.
 - Write **2KB of data** at that position.
 - Call `fsync()` after each write.
- The execution time of each workload must be measured independently.
 - Use `gettimeofday()` (or a similar high-resolution timing function).
 - The timing interval should include only the workload execution.
 - Exclude file creation time from the measurement.
 - Report the total elapsed time for each workload.

Outline

- File I/O
- C-library : fopen() 、 fread() 、 fwrite() 、 fclose() 、 fseek()
- Homework Assignment #1_1
- System call : open() 、 read() 、 write() 、 close() 、 lseek()
- Homework Assignment #1_2
- **Memory-mapped file**
- Homework Assignment #1_3
- Homework Assignment #1_4

Memory-mapped file

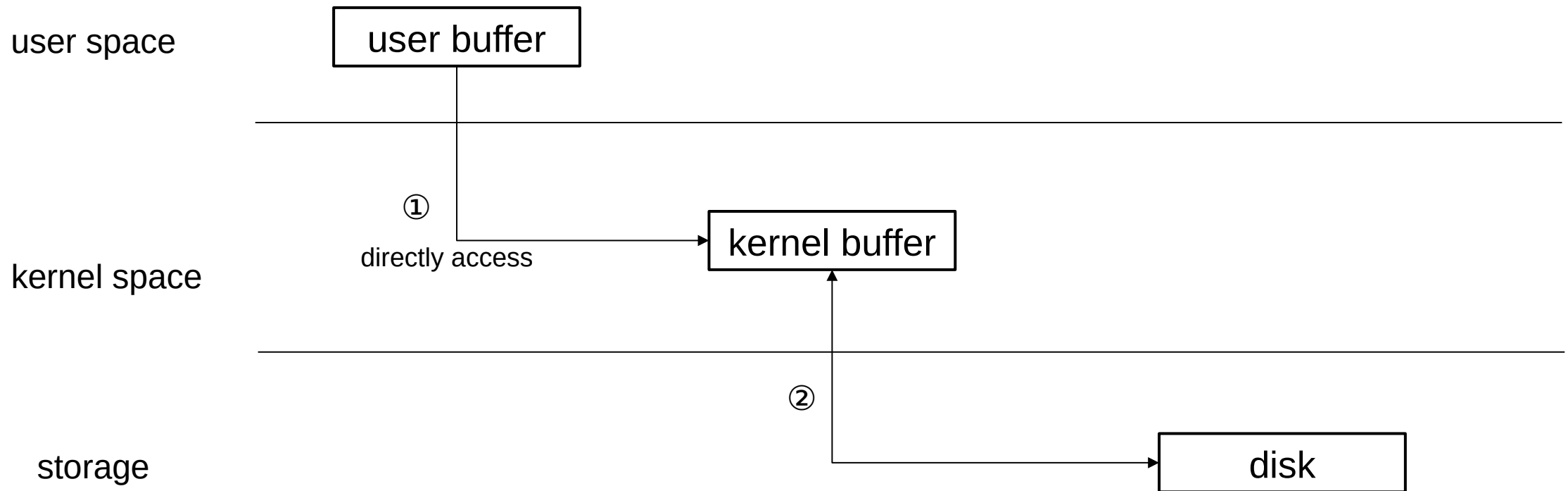
- Consider a sequential read of a file on disk using the standard system calls **open()**, **read()**, and **write()**. Each file access requires a system call and disk access.



Read system call

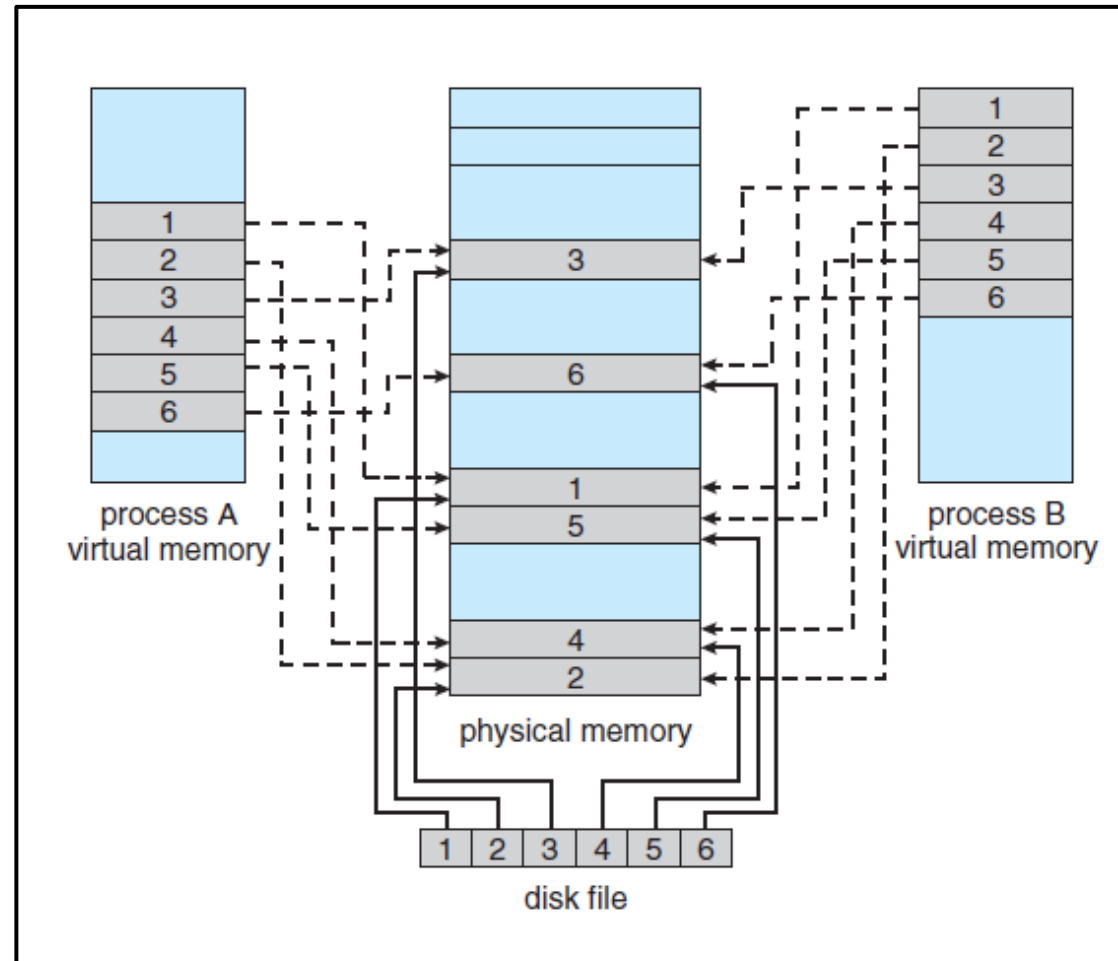
Memory-mapped file(cont.)

- We can use the virtual memory techniques to treat file I/O as routine memory accesses. This approach, known as **memory mapping** a file, allows a part of the virtual address space to be logically associated with the file.



memory-mapped read

Memory-mapped file(cont.)



Memory-mapped file(cont.)

■ mmap()

```
#include <sys/mman.h>
```

```
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
```

```
EX: char *map_f1;  
    map_f1 = mmap(NULL, filesize, PROT_READ, MAP_SHARED, f1, 0);
```

addr 、 **length** 、 **fd** 、 **offset**: The pages starting at **addr** and continuing for at most **length** bytes to be mapped from the object described by **fd**, starting at byte offset **offset**.

prot:

PROT_EXEC	Pages may be executed.
PROT_READ	Pages may be read.
PROT_WRITE	Pages may be written.
PROT_NONE	Pages may not be accessed.

Memory-mapped file(cont.)

■ mmap()

flags:

MAP_SHARED	Share this mapping with all other processes that map this object. Storing to the region is equivalent to writing to the file. The file may not actually be updated until msync(2) or munmap(2) are called.
MAP_PRIVATE	Create a private copy-on-write mapping. Stores to the region do not affect the original file. It is unspecified whether changes made to the file after the mmap() call are visible in the mapped region.

return value: On success, **mmap()** returns a pointer to the mapped area. On error, the value **MAP_FAILED** (that is, (void *) -1) is returned, and **errno** is set appropriately.

Memory-mapped file(cont.)

■ munmap()

```
#include <sys/mman.h>
```

```
int munmap(void *addr, size_t length);
```

```
EX: munmap(map_f1, filesize);
```

addr 、 **length**: The function remove any mappings for those entire pages containing any part of the address space of the process starting at **addr** and continuing for **len** bytes.

return value: Upon successful completion, it shall return 0; otherwise, it shall return -1 and set **errno** to indicate the error.

Memory-mapped file(cont.)

■ Example program

```
#include <sys/mman.h> //mmap
#include <sys/types.h> //
#include <sys/stat.h> //open()
#include <fcntl.h> //
#include <unistd.h>
#include <stdio.h>
#include <sys/time.h>
#include <string.h> //memcpy()

#include <errno.h>
int main(){
    char buffer[4];
    int f1;
    char *map_f1; //mapping param.

    f1 = open("hello.txt", O_RDWR); //open file you want to mmap
    if(f1 == -1)
    {
        printf("open hello.txt fail.\n");
        return 1;
    }
    //get the size of file
    struct stat st;
    stat("hello.txt", &st);
    long int filesize = st.st_size;
    printf("file size is: %ld\n", filesize);
```

Memory-mapped file(cont.)

■ Example program

```
//mmap
map_f1 = mmap(NULL, filesize, PROT_WRITE | PROT_READ, MAP_SHARED, f1, 0);
if (map_f1 == MAP_FAILED)
{
    printf("mmap1 error: %s\n", strerror(errno));
    return 0;
}
printf("map_f1 is: %ld\n", map_f1);
//read from the map you can also use it like a array buffer[0]=map_f1[0]
memcpy(buffer, map_f1, 4);
//you can change the offset by increasing it value
map_f1 += 4;
//write to map
memcpy(map_f1, buffer, 4);

map_f1 -= 4;
if(munmap(map_f1, filesize) == -1)
{
    printf("munmap_f1 failed.\n");
}

if(close(f1) == -1)
{
    printf("close f1 error.\n");
}

printf("end\n");
return 0;
```

```
}
```

Outline

- File I/O
- C-library : fopen() 、 fread() 、 fwrite() 、 fclose() 、 fseek()
- Homework Assignment #1_1
- System call : open() 、 read() 、 write() 、 close() 、 lseek()
- Homework Assignment #1_2
- Memory-mapped file
- **Homework Assignment #1_3**
- Homework Assignment #1_4

Homework Assignment #1_3

- Write a C program that creates a **100MB file** on your local disk and measures the execution time of five different operations using the **memory-mapped I/O interface**.
- Implement each of the following operations as a separate function
 - ☐ Sequential Read
 - ☐ Sequential Write
 - ☐ Random Read
 - ☐ Random Buffered Write (Single Sync)
 - ☐ Random Synchronous Write

Homework Assignment #1_3

■ Sequential Read

- ☐ Read the file sequentially from beginning to end.
- ☐ Read **4KB of data per operation**.
- ☐ Continue until the entire 100MB file has been read.

■ Sequential Write

- ☐ Overwrite the file with **100MB of new data** using sequential writes.
- ☐ Write **2KB of data per operation** from beginning to end.
- ☐ After completing all writes, call `fsync()` to ensure that the data is flushed to disk.

Homework Assignment #1_3

■ Random Read

- Repeat the following operation **50,000 times**:
 - Select a **4KB-aligned offset** uniformly at random within the file.
 - Access the selected offset via the memory-mapped region.
 - Read **4KB of data** from that position.

■ Random Buffered Write (Single Sync)

- Repeat the following operation **50,000 times**:
 - Select a **4KB-aligned offset** uniformly at random within the file.
 - Access the selected offset via the memory-mapped region.
 - Write **2KB of data** at that position.
- After all writes have been completed, call `fsync()` once.

Homework Assignment #1_3

- Random Synchronous Write
 - Repeat the following operation **50,000 times**:
 - Select a **4KB-aligned offset** uniformly at random within the file.
 - Access the selected offset via the memory-mapped region.
 - Write **2KB of data** at that position.
 - Call `fsync()` after each write.
- The execution time of each workload must be measured independently.
 - Use `gettimeofday()` (or a similar high-resolution timing function).
 - The timing interval should include only the workload execution.
 - Exclude file creation time from the measurement.
 - Report the total elapsed time for each workload.

Outline

- File I/O
- C-library : fopen() 、 fread() 、 fwrite() 、 fclose() 、 fseek()
- Homework Assignment #1_1
- System call : open() 、 read() 、 write() 、 close() 、 lseek()
- Homework Assignment #1_2
- Memory-mapped file
- Homework Assignment #1_3
- **Homework Assignment #1_4**

Homework Assignment #1_4

- Analyze and compare the performance results obtained from Homework #1_1, #1_2, and #1_3.
- Provide a detailed explanation of the observed differences among the three I/O interfaces.
- **Experimental Requirement**
 - The page cache must be cleared after each program run to ensure fair and consistent measurements.

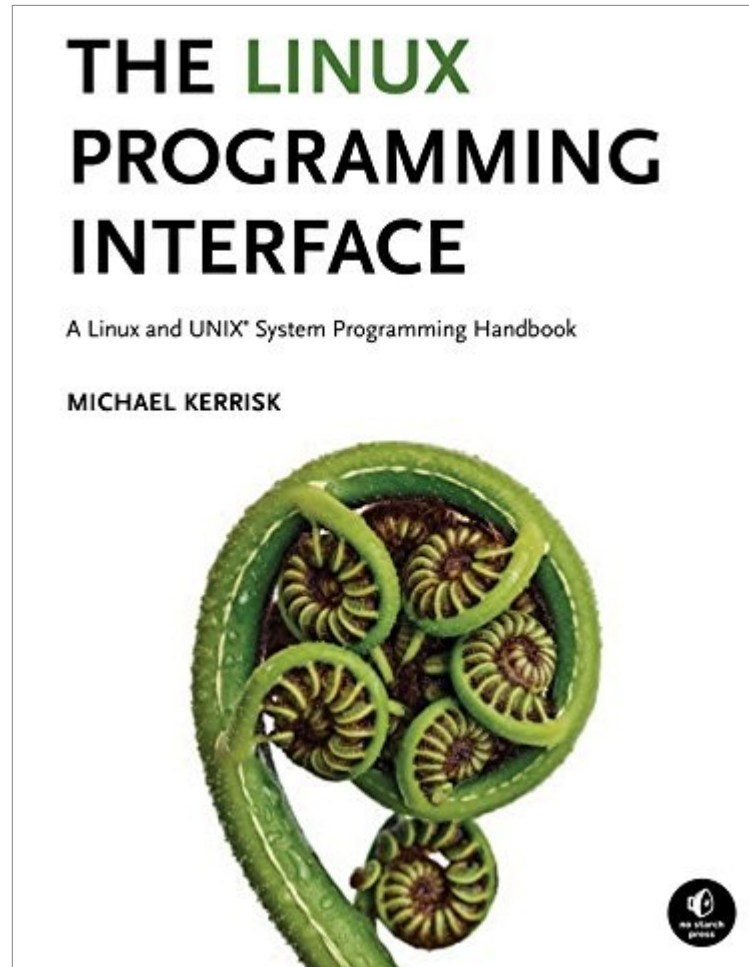
```
willy@oslab: ~/FS/HW1
willy@oslab:~$ cd FS
willy@oslab:~/FS$ cd HW1
willy@oslab:~/FS/HW1$ ./HW1_1
--- Standard I/O Benchmark (Homework 1_1) ---
1. Sequential Read:      0.4832 sec
2. Sequential Write:    0.1513 sec
3. Random Read:         0.0783 sec
4. Random Buffered Write: 0.8849 sec
5. Random Sync Write:   299.9847 sec
willy@oslab:~/FS/HW1$
```

```
willy@oslab: ~/FS/HW1
willy@oslab:~$ cd FS
willy@oslab:~/FS$ cd HW1
willy@oslab:~/FS/HW1$ ./HW1_2
--- System Call Benchmark (Homework 1_2) ---
1. Sequential Read:      0.6554 sec
2. Sequential Write:    0.3906 sec
3. Random Read:         0.0826 sec
4. Random Buffered Write: 0.8356 sec
5. Random Sync Write:   304.1507 sec
willy@oslab:~/FS/HW1$
```

```
willy@oslab: ~/...
willy@oslab:~$ cd FS
willy@oslab:~/FS$ cd HW1
willy@oslab:~/FS/HW1$ ./HW1_3
1. Sequential Read: 0.5786 sec
2. Sequential Write: 0.2615 sec
3. Random Read: 0.0249 sec
4. Random Buffered Write: 1.1013 sec
5. Random Sync Write: 172.0529 sec
willy@oslab:~/FS/HW1$ SS
```

Reference

- The Linux Programming Interface: A Linux and UNIX System Programming Handbook.



Turn in

- Deadline
2026/4/1 23:30:00
- Upload to i-learning
- File name
 - HW1_ID.zip (e.g. HW1_7113056017.zip)
 - HW111.c 、 HW112.c 、 HW113.c
 - Report file (Word format)

TA

- Name: 陳立崙
- Email: g114056113@smail.nchu.edu.tw
 - Title format: HW1 - [your name]
- Lab: OSNET(1001A)